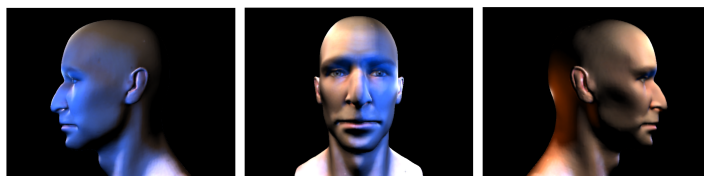


Subsurface Scattering in Screen Space Report

Jessica Robbins

March 2026



1 Introduction

Subsurface Scattering is the process in which light enters a surface and bounces around before leaving. It results in a translucent effect that can be seen in tree leaves, wax, soap, fruit, etc. Human skin, however, is particularly interesting because not only does the light enter and bounce around through the surface, but the skin is made up of many layers of translucent material. Furthermore, the human eye is exceptionally perceptive of this difference, so even if the skin simulation is only slightly off, it will still be noticeable. So, as a result the human skin is quite difficult to render in a way that is efficient and realistic.

In computer graphics, human skin is often used in fields like cinematography and game development. For cinematography, it is much simpler to create realistic skin because there is the luxury of time. While, in systems like video games, the skin needs to be rendered in real-time. This creates a challenge for these applications to create realistic looking skin and so previously the process was to simply ignore the translucent aspect of the skin entirely.

This is where Jorge Jimenez, Veronica Sundstendt and Diego Gutierrez saw an opportunity to advance the current methods. The goal is to achieve the effects of subsurface scattering in a way that is good enough to be perceived as realistic, but is also computationally efficient enough for video games. After researching real-time algorithms they found a common theme that blurring of high-frequency details results in believable simulation. These methods were done in texture space and were computationally efficient, however they lacked the ability to scale well. While one model could be rendered without much cost, adding more assets would mean more textures and so the scene would quickly become expensive.

The authors propose a different approach which is to simulate the scattering effects in screen space. Not only does this approach offer scalability because it reduces the problem of simulating transparency to a post process, it is easy to integrate in any game engine. The possible pitfall in their proposal is that since the algorithm is in screen space there is not as much access to information. They go on to study how much of an affect this lack of information has on the results by comparing it to the results of a) no subsurface scattering effect, b) state-of-the-art, texture-space algorithm, c) a naive mapping of an existing algorithm into screen space. In the end their method was not only faster and more scalable, the comparison with the state-of-the-art results was not in fact very drastic, proving that their proposal is in fact a valid way to render realistic looking skin efficiently.

2 My Set Up

In order to implement their method, I started with the code from assignment four and added a file reader to import a head model. I needed to get the inputs that they have: depth, matte, specular and diffuse color. It is important to note that the specular light is not used in the SSSS calculations but added to the final image after running the algorithm on the other inputs. It is still calculated in the first pass, though. Each of these textures are output by the fragment shader using Multiple Render Targets (MRT).

The diffuse color is the texture color which was achieved by importing the texture png and passing that, the uv coordinates and sampler to the GPU.

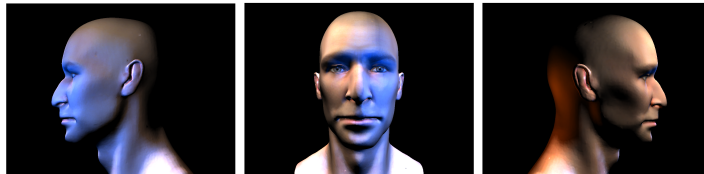


Figure 1: Face model with applied texture.

The matte of the object represents which parts of the model are made up of skin material or not, hence why the matte is rendered as black and white for 0 or 1 in Figure 2. The matte was simple for both my implementation and theirs because they were bald heads, so all visible aspects were skin. I chose to include the eyes in the skin material because the model was not set up to distinguish materials in this way.



Figure 2: Face model displaying only matte.

The specular texture was also captured in the first render pass. I attempted to use the Kelemen/Szirmay-Kalos model [KSK01], but I believe my implementation of this was incorrect as the specular light turns into a cone point when facing the incoming light. However, I chose not to stress on this as much because for the most part it seemed correct and I wanted to ensure I had enough time for the actual SSSS algorithm.

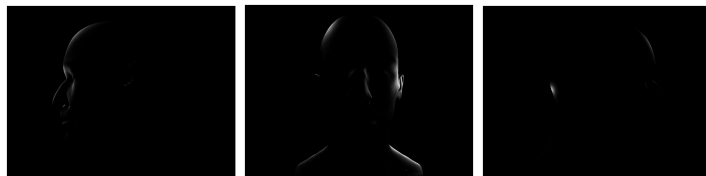


Figure 3: Face model displaying only specular.

The depth was the most difficult input parameter to render because while the depth buffer is written to during the original render pipeline, it is not possible to read from it and write at the same time. So, it was necessary to create two render passes with two pipelines, two binding groups and two shader codes. Additionally, while the paper specifies that they linearized the depth using the method from [Gil06] that implementation was difficult for me to reproduce. I am not sure why, but I found that this equation worked while the other one did not. $\text{linear} = (2.0 * \text{near}) / (\text{far} + \text{near} - d * (\text{far} - \text{near}))$; These two are very similar in that they both calculate the depth from the viewing perspective rather than from the normalized device coordinates. So, idealistically they should achieve the same results.



Figure 4: Linearized depth of face model.

3 Screen Space Algorithm

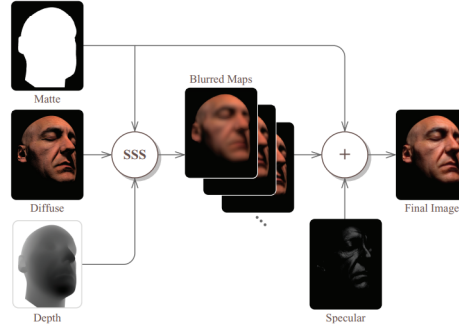


Figure 5: Overview of their screen-space algorithm.

The main algorithm of the program is to take the above inputs: matte, diffuse and depth and achieve the subsurface scattering effect by blurred maps and add the specular component to create the final image as you can see in Figure 5. The key point that the authors are trying to make is that the subsurface scattering effect can be done by blurring the diffuse in screen space. They create the blurred maps by combining two different ways of applying the convolution kernel. The first way is to use a single bi-dimensional convolution with jittered samples as proposed in [HBH09] and the second way is to use the six one-dimensional Gaussians described in [dLE07]. They then combine these two using a weighted sum in a second pass. Finally, to create the final product they add the specular component that was calculated using the Kelemen/Szirmay-Kalos model [KSK01].

After getting the matte, diffuse and depth from the first pass, I applied the convolution kernel in the second pass. (Although, technically in my implementation this is the third pass because I linearized and displayed the depth in an additional pass.) Since in the set up I used MRT to output the three textures, I bound them and passed them to the shaders for utilization in the next pass.

Implementing the single bi-dimensional convolution with jittered samples and using a sum of Gaussian kernels, was quite simple as they provided a simple code structure to follow. How it works is that the convolution is basically a filter that is used on the image with the Gaussians being what blur the image. The kernel is a grid from which samples multiple neighboring pixels and combines them to make the final color of the current pixel. The jittered samples allow for perturbing sample positions instead of uniform. I accessed the color from the diffuse texture and the depth from the depth texture. Adding to their implementation, I computed depth gradients and used them to create stretch values based on the following equations provided in the paper. These stretch values changed the size of the kernel according to depth. This is because further away pixels and large gradients in the depth map should both use narrower kernels.

$$s_x = \frac{\alpha}{d(x, y) + \beta \cdot \text{abs}(\nabla_x d(x, y))},$$

$$s_y = \frac{\alpha}{d(x, y) + \beta \cdot \text{abs}(\nabla_y d(x, y))},$$

Above, $d(x, y)$ is the depth at pixel (x, y) . The gradients ∇_x and ∇_y are computed using `ddx` and `ddy`. α controls overall subsurface scattering, while β modulates it based on depth variation. Those two values were very important to the final product because if they are too small the blur affect is minimal and the subsurface scattering does not seem apparent. If alpha was set too high, the image looked far too blurry and if the beta was increased at some point the depth gradient lines started showing on the image. High values were chosen to display these changes:

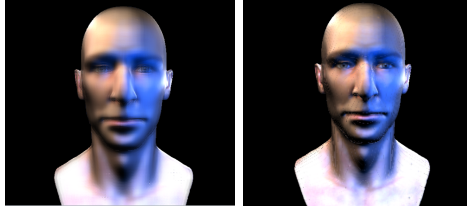
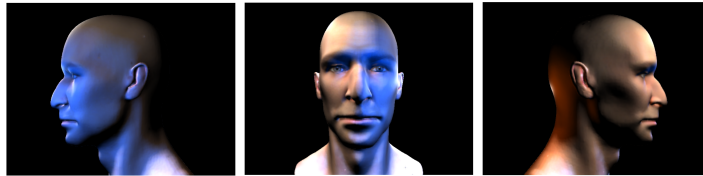


Figure 6: left: alpha = 50 and beta = 800, right: alpha = 11 and beta = 9000

In the end I chose to increase alpha to 16 because the model's texture was already so smooth. This slight increase allowed the blur to be more visible.

I used the mix feature in WGS� to only apply the blur maps to the skin material sections of the model.

4 Conclusion



You can see that the resulting images have the applied blur. Although, this model does not show it very well because the skin is already very smooth for this model. A better representation for the method would have been shown on a model with a more accurate and detailed skin texture. Additionally, I did not isolate out the eyes in matte as I was worried it was outside the scope of this project, but that is an additional add-on to this project because the eyes

themselves are not skin so they should not be blurred. They also applied a bloom filter at the end that I did not have time to implement.

Finally, over all through their algorithm I was able to successfully recreate the subsurface scatter in screen space.

References

- [dLE07] Eugene d'Eon, David Luebke, and Eric Enderton. Efficient rendering of human skin. In *Proceedings of the Eurographics Symposium on Rendering*, 2007.
- [Gil06] David Gillham. Real-time depth-of-field implemented with a postprocessing-only technique. In Wolfgang Engel, editor, *ShaderX5*, chapter 3.1, pages 163–175. Charles River Media, 2006.
- [HBH09] John Hable, George Borshukov, and Jan Kautz Hejl. Fast skin shading. In Wolfgang Engel, editor, *ShaderX7*, chapter 2.4, pages 161–173. Charles River Media, 2009.
- [KSK01] Csaba Kelemen and László Szirmay-Kalos. A microfacet based coupled specular-matte brdf model with importance sampling. In *Proceedings of the Eurographics Short Presentations*, 2001.